apply tail-call optimization, my advice is to write the iterative version of these methods, to avoid stack overflow.

Listing 6.2 method search

```
function search(node,
s)                                                    #1
  if s == “” then
                                                    #2
    return
node.keyNode
    #3
  c, tail ←
s.splitAt(0)
   #4
  if node.children[c] == null then
                            #5
    return false
  else
    return search(node.children[c],
tail)                                      #6
```

#1 Method `search` takes a trie node and the string key `s` to be searched; it returns `true` if `s` is stored in the trie, or `false` otherwise. We assume that `node` is never `null`: this is a reasonable assumption if this method is implemented as a private method, internally called by the trie’s API `search` method.

#2 Check if the string searched is empty: if it is, since this method is implemented using recursion, we know it has traversed the whole path in the trie.

#3 We are at the target node for the target key: the trie stores it only if current node is a key node.